

Title

Inhalt

Clock

Unter der Clock versteht man den Impulsgeber eines Prozessors. Mit jedem Impuls der Clock führt der Prozessor einen Schritt aus. Dadurch gibt die Clock auch die Geschwindigkeit des Prozessors vor. Je schneller die Clock, desto schneller der Prozessor. Man kann die Geschwindigkeit der Clock allerdings nicht unbegrenzt erhöhen, da die restlichen Komponenten des Prozessors nur begrenzt schnell arbeiten können. Man muss daher die schnellstmögliche Frequenz finden, bei der die restlichen Komponenten weiterhin zuverlässig arbeiten. Außerdem steigt mit der erhöhten Rechengeschwindigkeit auch der Strombedarf des Prozessors. Eine geringere Frequenz kann also vorteilhaft sein, wenn man Energie sparen möchte, zum Beispiel weil die maximale mögliche Energieaufnahme begrenzt ist. Als Clock-Signal kann grundsätzlich jedes Rechtecksignal verwendet werden. Üblicherweise verwendet man jedoch einen Quarz, wodurch die Impulse eine sehr genaue Frequenz haben. Der Prozessor kann anhand der Zyklen die genau vergangene Zeit bestimmen. Ein Quarz wird aber nicht zwingend gefordert. Häufig besitzen Mikroprozessoren eine eingebaute RC-Schaltung, die zur Erzeugung des Clock-Impulses verwendet werden kann. Diese ist jedoch nicht so genau wie ein Quarz. (Sie kann sich z. B. bei Temperaturänderungen verändern.) Theoretisch ist es sogar möglich, einen Taster zu verwenden, wodurch eine Art Schrittbetrieb erreicht wird. Das ist aber lediglich für experimentelle/forschende Zwecke sinnvoll. Zudem haben moderne Prozessoren eine Mindestgeschwindigkeit und arbeiten nicht mehr richtig, wenn die Clock zu langsam ist. [2, p. 93, 158]

Programmzähler

Der Programmzähler enthält die Speicheradresse der nächsten, auszuführenden Instruktion. Am Anfang eines Instruktionszyklus wird die Instruktion an der von dem Programmzähler vorgegebenen Speicheradresse in das Steuerwerk geladen. Anschließend wird der Programmzähler um 1 erhöht, sodass er auf den nächsten Wert im Speicher zeigt. Manche Instruktionen sind größer als eine Speichereinheit. In dem Fall wird der Programmzähler mehrfach innerhalb eines Zyklus inkrementiert. Sprunginstruktionen (JMP, RET, BNE, etc.) funktionieren, indem die Zieladresse der Instruktion in den Programmzähler geladen wird [2, p. 179]. Im nächsten Schritt wird also nicht mehr die „nächste“ Instruktion abgerufen, sondern die an der angegebenen Zieladresse. *Note: Bei einer return-Instruktion wird die Zieladresse vom Stack gepoppt.*

Bus

Ein Bus verbindet die einzelnen Komponenten eines Prozessors miteinander. Welche Komponente gerade auf den Bus ausgibt und welche von dem Bus liest, wird durch das Steuerwerk bestimmt.

- **Datenbus:** Über diesen Bus werden Daten transferiert.
- **Addressbus:** Über diesen Bus werden Speicheradressen übermittelt. Die Größe dieses Busses gibt an, wie viel Speicher von dem Prozessor direkt angesprochen werden kann. So kann ein Prozessor mit einem 16-Bit-Addressbus maximal 65.535 Speichereinheiten ansprechen.

Daten und Adressen können den gleichen physischen Bus verwenden. Das verringert allerdings die Geschwindigkeit des Prozessors, da der Bus immer abwechselnd für Adressen und Daten eingesetzt werden muss. [2, p. 225]

Speicher

Als Speicher versteht man den gesamten Speicher eines Prozessors. Dies umfasst sowohl den Programmspeicher (ROM), der die ausführbaren Instruktionen enthält, als auch den Arbeitsspeicher (RAM),

den der Prozessor zum Speichern temporärer Daten verwendet und der meist flüchtig ist. Der Speicher wird über Speicheradressen adressiert. Bei der Verarbeitung von Speicheradressen kommt häufig Schaltlogik zum Einsatz. Beispiel: Angenommen, ein Prozessor hat eine Adressgröße von 8 Bit, dann könnte der Prozessor insgesamt 256 Werte adressieren. Man kann nun einen RAM- und einen ROM-Chip so an den Prozessor anschließen, dass der RAM-Chip aktiv ist, wenn das MSB der Adresse HIGH ist. Wenn das Bit LOW ist, wird stattdessen der ROM-Chip aktiviert. Die ersten 128 Werte kommen dann aus dem ROM-Chip, die Werte an den Adressen 128–255 aus dem RAM-Chip. Der Prozessor selbst bekommt davon aber nichts mit. Der vorhandene Speicher hängt vom Prozessor ab. Heutige Mikrocontroller haben üblicherweise einen integrierten RAM- und Flash-Speicher. Letzterer dient als Programmspeicher. Mikroprozessoren (z. B. x86) haben stattdessen lediglich einen Arbeitsspeicher. Das Programm, das ausgeführt werden soll, wird vor dem Ausführen aus einem externen Speicher in den RAM geladen.

Register

Die Register sind die schnellste Speicherart in einem Prozessor. Sie werden direkt über eigene Instruktionen angesprochen und müssen somit nicht durch den üblichen Speicheradressierungsprozess. Sie halten Daten für die unmittelbare Verwendung oder zur Steuerung des Prozessors. So können bei einem ATmega durch die Register DDRx, PINx und PORTx die I/O-Pins des Prozessors gesteuert werden [7, p. 59]. Andere Register dienen wiederum als Eingabe für die ALU [7, p. 10].

Je nach Prozessor können Register auch eine Speicheradresse haben, auch wenn sie nicht physisch im RAM liegen [7, 9. 12].

Stack

Der Stack ist ein Speicherbereich, der nach dem FIFO (FirstIn-FirstOut)-Prinzip arbeitet. Neue Daten werden immer nur an den Anfang des Stacks angefügt und auch immer nur vom Anfang des Stapels entnommen. Bewerkstelligt wird dies durch das *Stackpointer*-Register, welches auf den Anfang (neuestes Element) des Stacks zeigt. Der Stack befindet sich im Arbeitsspeicher des Prozessors. Um ein Element dem Stack hinzuzufügen, wird der Stackpointer inkrementiert und das Datum an die neue, vom Stackpointer vorgegebene Adresse geschrieben. Um ein Element aus dem Stack zu entfernen, wird das Datum aus der von dem Stackpointer vorgegebenen Adresse ausgelesen und der Stackpointer danach dekrementiert. Das Programm greift entweder explizit durch PUSH- und POP-Instruktionen auf den Stack zu oder implizit durch Instruktionen wie CALL und RET (return). [2, p. 229]

Note: Je nach Architektur kann der Stackpointer auch auf das nächste freie Element zeigen, anstatt wie beschrieben auf das neueste. Außerdem ist es nicht unüblich, dass der Stackpointer bei einem push dekrementiert statt inkrementiert wird.

Vektoren

Vektoren sind spezielle Adressen, die der Prozessor in bestimmten Situationen anspringt. Hierzu gehören unter anderem der Start- oder Resetvektor, der bei einem Reset angesprungen wird und somit der Einstiegspunkt des Programms ist, sowie die Interruptvektoren, die durch bestimmte Auslöser angesprungen werden und den normalen Programmablauf unterbrechen. Solche Interrupts können unter anderem durch Timer oder externe Signale (GPIO) ausgelöst werden. Die Adressen unterscheiden sich je nach Prozessor. So ist der Resetvektor bei einem ATmega (z. B. Arduino) an Adresse \$0000, während er bei einem 6502 an Adresse \$FFFFFFFFFFFFC liegt. Da die Vektoren meist direkt aufeinander folgen, bietet ein Vektor lediglich Platz für eine einzige Instruktion, mit der dann zu der tatsächlichen Routine gesprungen wird. [5] [6]

Rechenwerk

Das Rechenwerk (auch ALU - Arithmetic Logic Unit) führt arithmetische und logische Operationen aus. In modernen Prozessoren kann man üblicherweise Register angeben, die verrechnet werden sollen. Das Ergebnis wird häufig in eines der Ausgangsregister geschrieben. Es gibt aber auch Architekturen, bei denen man ein Ausgangsregister angeben kann [1]. Bei älteren Prozessoren kann man die Eingaberegister üblicherweise nicht frei wählen. Hier gibt es ein Akkumulator-Register (Akku / A), welches als festes Ein- und Ausgaberegister fungiert. Je nach Architektur kann man entweder das zweite Register auswählen oder es gibt ein festes B-Register für den zweiten Operanden [2, S. 142].

Steuerwerk

Ein Instruktionszyklus besteht aus 3 Phasen.

- Fetch: Die nächste Instruktion wird aus der Adresse, die der Programmzähler angibt, in das Instruktionsregister geladen.
- Decode: Das Steuerwerk dekodiert die geladene Instruktion. Das heißt, dass die Instruktion, welche als binäre Zahl im Instruktionsregister liegt, in ein einzelnes Signal für jede mögliche Instruktion aufgeteilt wird. (Beispiel: Ist die ADD <0x87>-Instruktion geladen worden, schaltet der Decoder die ADD-Signalleitung ein und alle anderen aus.) [2, p. 158]. Dieses Signal wird nun durch die Steuermatrix und mithilfe eines Ringzählers weiterverarbeitet. Ein Ringzähler ist ein Zähler, der nicht binär zählt, sondern seine Ausgänge der Reihe nach einschaltet. Nach der letzten Zahl beginnt der Zähler wieder mit der ersten Zahl. Es ist immer nur ein Ausgang aktiv [2, S. 159]. Die Steuermatrix decodiert die Instruktion plus Ringpuffer in die einzelnen Steuersignale der CPU [2, p. 161].
- Execute: Mit jedem Schritt der Clock wird der Ringbuffer inkrementiert, wodurch die Instruktion schrittweise ausgeführt wird.

Hinweis: Die Größe und Komplexität der Decoder und Steuermatrix wächst mit einer steigenden Anzahl der Instruktionen stark an. Dies ist auf einem Chip kein Problem. Implementiert man einen Prozessor aber auf einem Breadboard, wird dies schnell unhandlich. Stattdessen kann man hier einen EEPROM verwenden. In dem Fall gibt man den Zustand des Zählers (binärer Zähler anstelle eines Ringzählers) und die Instruktion als Adresse in den EEPROM. Der EEPROM muss vorher so programmiert werden, dass die Steuersignale entsprechend geschaltet werden.

Optimierungen

Pipelining

Die Teile des Prozessors, auf die die Fetch-, Decode- und Execute-Schritte zugreifen, sind unabhängig voneinander. Daher gibt es keine Konflikte, wenn die Schritte parallel ausgeführt würden. Diesen Prozess nennt man Pipelining. Während eine Instruktion ausgeführt wird, wird bereits die nächste Instruktion decodiert und die übernächste geladen. [3]

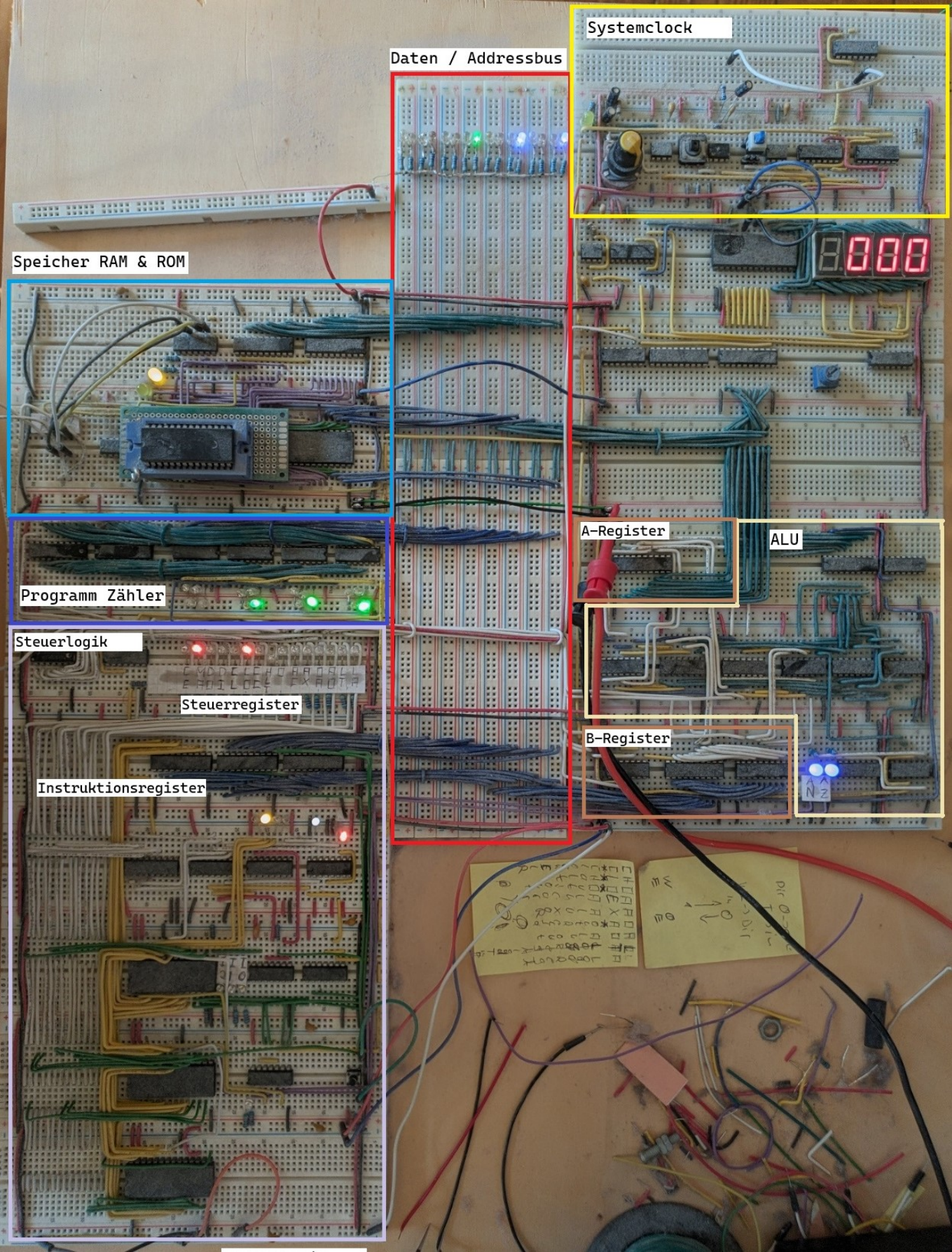
Reordering

Die Anzahl an Zyklen, die eine CPU benötigt, um eine Instruktion auszuführen, variiert je nach Instruktion. Moderne Prozessoren optimieren den Programmablauf, in dem sie die Reihenfolge der Instruktionen verändern. So wird eine zeitaufwändige Instruktion früher gestartet, vorausgesetzt die erforderlichen Parameter für die Instruktion sind bereits vorhanden.

Referenz Architekturen

Die heutigen Prozessoren nutzen üblicherweise die von-Neumann- oder die Harvard-Architektur. Die beiden Architekturen unterscheiden sich primär darin, wie sie Programm- und Datenspeicher ansprechen. Bei der von-Neumann-Architektur liegen Programm und Daten im selben Speicher. Die Harvard-Architektur trennt den Programm- und Arbeitsbereich und bindet sie über dedizierte Busse an. Dadurch ist der Prozessor in der Lage, Programm- und Datenspeicherabfragen parallel durchzuführen, wodurch der Prozessor schneller und effizienter ist. Außerdem ist es möglich, den Programmspeicher schreibgeschützt (ROM) anzubinden, wodurch der Prozessor nicht in der Lage ist, das Programm zur Laufzeit zu verändern, was unter anderem sicherheitstechnische Vorteile hat. Diese Architektur erfordert allerdings einen extrem hohen Aufwand, weswegen sie hauptsächlich in Echtzeitanwendungen wie Mikrocontrollern und Signalprozessoren Anwendung findet [4]. Die meisten Prozessoren (wie u. a. x86) verwenden die von-Neumann-Architektur aufgrund ihrer Einfachheit.

Beispielprozessor



[Abb. 1]

- **Speicher:** Stellt ROM (EEPROM, großer Chip in der Mitte) und RAM (leicht verdeckter Chip). Der angesprochene Chip wird durch das MSB der Adresse bestimmt.
 - **Daten / Adressbus:** Daten und Adressen liegen auf demselben Bus. Das reduziert den benötigten Platz, reduziert aber auch die Prozessorgeschwindigkeit. Der Bus ist 16 Bit breit, für Daten werden aber lediglich 8 Bit verwendet.
 - **Clock:** Die Clock wird entweder manuell über einen Knopf betätigt oder läuft automatisch, wobei die Geschwindigkeit über ein Poti geregelt werden kann. Das Taktsignal wird über einen gelben Draht an alle Komponenten des Prozessors geleitet.
 - **Steuerlogik:** Die Steuerlogik wurde über EEPROMs realisiert. Die Signale des Steuerregisters
-

Quellen

1. [elektronik-kompendium] <https://www.elektronik-kompendium.de/sites/com/1310171.htm> (Abgerufen: 10.04.2026 12:40)
2. Malvino, Brown (1995) Digital Computer Electronics: Third Edition, 40. reprint 2018. Indien. Mc Graw Hill Education.
3. [Computerphile - CPU Pipeline] <https://www.youtube.com/watch?v=BVNx3wtJ9vs> (Abgerufen: 10.04.2026 12:40)
4. [kreissl] <http://www.kreissl.info/ra.php> (Abgerufen: 03.04.2026 13:26) / archive.org
5. [microchip] (<https://onlinedocs.microchip.com/oxy/D-AAB4173A-6BB6-4A4B-A053-1ED838585692-en-US-4/GUID-EC3B67E3-BF11-4E9B-AB9D-8D20942E5434.I>) (Abgerufen: 09.04.2026 17:01)
6. [6502.org] <https://6502.org/users/andre/65k/af65002/af65002int.html#reset> (Abgerufen: 09.04.2026 10:46)
7. [microchip.com] https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf (Abgerufen: 10.04.2026 11:57) - ATmega datasheet

Bilder

1. Foto, Colin Meihöfer